TECHGAMER

2026

PROJETO
TECHGAMER

GERENCIADOR DE AVALIAÇÕES DE JOGOS

Edino Rogério V. Alves Jr
Fillipe Correa de A. Sales
Gabriel L. Colquehuanca
João Pedro V. de Miranda

SOBRE O TECHGAMER

O TechGamer é uma solução desenvolvida pela TechForce com o objetivo de facilitar a organização e a avaliação de jogos eletrônicos.

A proposta surgiu da necessidade de oferecer aos jogadores uma maneira simples de organizar seus jogos e registrar suas experiências, sem depender de plataformas externas.

O sistema funciona como uma biblioteca pessoal, permitindo que o usuário cadastre jogos, consulte suas informações, atribua notas de 0 a 10 e registre comentários sobre suas experiências.

Além disso, o TechGamer permite editar e excluir jogos e avaliações, pesquisar títulos, utilizar filtros de busca e consultar as médias das avaliações, mantendo as informações armazenadas localmente para consultas futuras.

Desenvolvido em Python, com Flask no servidor e Qt (PySide6) na interface gráfica, o TechGamer oferece uma experiência prática, organizada e acessível, com perfis locais e sem necessidade de autenticação ou compartilhamento de informações pela internet.

1. INTRODUÇÃO

O TechGamer é um projeto acadêmico desenvolvido com a finalidade de criar um sistema de gerenciamento de jogos e avaliações pessoais.

O aplicativo permite que o usuário organize uma biblioteca de jogos, registre informações sobre cada título e armazene avaliações contendo notas e comentários.

O projeto foi desenvolvido em Python, utilizando Flask para o servidor, Qt (PySide6) para a interface gráfica e arquivos JSON para armazenamento dos dados.

A aplicação utiliza uma arquitetura cliente-servidor, na qual a interface gráfica se comunica com o servidor Flask por meio de requisições HTTP, permitindo o gerenciamento das informações e sua persistência local.

Além das funcionalidades do aplicativo, o projeto demonstra a utilização de conceitos de programação, arquitetura cliente-servidor e padrões de projeto, incluindo Builder, Proxy e Observer, estudados na disciplina.

2. OBJETIVO GERAL

Desenvolver um aplicativo desktop para gerenciar uma biblioteca pessoal de jogos e suas avaliações, permitindo cadastrar, consultar, editar e excluir informações, utilizando Python, Flask, Qt (PySide6) e armazenamento local em arquivos JSON.

3. OBJETIVOS ESPECÍFICOS

- Desenvolver um servidor Flask para processar as operações do sistema.
- Criar uma interface gráfica utilizando Qt (PySide6).
- Permitir o cadastro, a consulta, a edição e a exclusão de jogos.
- Permitir o registro e o gerenciamento de avaliações e comentários opcionais.
- Implementar recursos de pesquisa, filtros e ordenação de jogos.
- Permitir a criação e utilização de perfis locais para organizar as avaliações.
- Calcular e apresentar a média das avaliações de cada jogo.
- Validar as informações fornecidas pelo usuário.
- Armazenar os dados em arquivos JSON.
- Manter os dados disponíveis após o encerramento e a reinicialização do aplicativo.
- Estabelecer a comunicação entre cliente e servidor por meio de requisições HTTP.
- Aplicar e documentar os padrões de projeto Builder, Proxy e Observer.
- Realizar testes automatizados e manuais para verificar o funcionamento do sistema.

4. DESCRIÇÃO E FUNCIONAMENTO DO SISTEMA

O TechGamer funciona como uma biblioteca pessoal de jogos, permitindo ao usuário organizar seus títulos e registrar avaliações de maneira simples e prática.

Ao abrir o aplicativo, o usuário tem acesso a uma interface gráfica que apresenta os jogos cadastrados e disponibiliza recursos de pesquisa, filtros, consulta de informações e gerenciamento de jogos e avaliações.

O sistema também permite utilizar perfis locais para identificar os autores das avaliações, sem necessidade de login ou autenticação.

4.1. CADASTRO DE JOGOS

O usuário pode cadastrar jogos informando:

- Nome do jogo.
- Gênero.
- Plataforma.
- Ano de lançamento.
- Descrição, quando desejar.
- Capa do jogo, opcionalmente.

Cada jogo recebe um identificador único gerado automaticamente pelo sistema, permitindo sua consulta, edição e exclusão.

4.2. CONSULTA DE JOGOS

O aplicativo exibe os jogos registrados na biblioteca.

O usuário pode pesquisar jogos, aplicar filtros e selecionar um título para visualizar suas informações e avaliações. Caso ainda não existam jogos cadastrados, o sistema apresenta uma mensagem informativa.

4.3. EDIÇÃO E EXCLUSÃO DE JOGOS

O usuário pode modificar as informações dos jogos cadastrados.

Também pode excluir um jogo de sua biblioteca.

Quando um jogo é excluído, suas avaliações associadas também são removidas, conforme a regra prevista no contrato da API.

A interface solicita confirmação antes da exclusão, evitando remoções acidentais.

4.4. CADASTRO DE AVALIAÇÕES

Após selecionar um jogo, o usuário pode registrar uma avaliação contendo:

- Nota numérica de 0 a 10.
- Comentário opcional.

O sistema aceita notas com casas decimais, como 8,5 ou 9,5.

Cada avaliação fica associada ao jogo correspondente e ao identificador do perfil local utilizado.

4.5. CONSULTA E GERENCIAMENTO DE AVALIAÇÕES

O usuário pode visualizar as avaliações cadastradas de cada jogo.

Também pode editar a nota e o comentário de suas avaliações ou excluí-las.

Um jogo pode possuir mais de uma avaliação, realizadas por diferentes perfis locais.

O sistema disponibiliza a seção "Minhas avaliações", permitindo ao usuário consultar e gerenciar as avaliações associadas ao seu perfil.

Embora não exista autenticação por login e senha, o sistema utiliza identificadores locais e aplica regras de permissão para edição e exclusão de avaliações.

4.6. ARMAZENAMENTO DAS INFORMAÇÕES

Os jogos e as avaliações são armazenados localmente em arquivos JSON.

Dessa forma, os dados permanecem disponíveis após fechar e abrir novamente o programa. O armazenamento é realizado pelo servidor Flask, por meio de um módulo específico de persistência.

5. ESCOPO DO PROJETO

O TechGamer foi desenvolvido com foco no gerenciamento local de jogos e avaliações, considerando os objetivos acadêmicos e as funcionalidades definidas pela equipe.

Nesta versão, o sistema não possui:

- Autenticação por login e senha.
- Compartilhamento de avaliações pela internet.
- Integração com lojas de jogos.
- Integração com APIs externas.
- Banco de dados.
- Serviços de nuvem ou sincronização online.

O aplicativo não tem como finalidade funcionar como uma loja de jogos, rede social ou plataforma de distribuição de jogos.

Essas limitações correspondem ao escopo definido pela equipe, considerando os requisitos da disciplina e o prazo disponível para o desenvolvimento.

6. TECNOLOGIAS UTILIZADAS

6.1. PYTHON

Linguagem de programação utilizada no desenvolvimento do sistema.

6.2. FLASK

Framework utilizado para desenvolver o servidor responsável por receber requisições HTTP, validar informações, executar operações e retornar respostas ao cliente.

6.3. QT

Tecnologia utilizada para construir a interface gráfica do aplicativo desktop.
A interface apresenta telas, formulários, botões, filtros, avaliações e mensagens para interação com o usuário.

6.4. JSON

Formato utilizado para armazenar localmente as informações dos jogos e avaliações, garantindo a persistência dos dados após o encerramento da aplicação.

6.5. HTTP

Protocolo utilizado na comunicação entre a interface gráfica e o servidor Flask, permitindo o envio de requisições e o recebimento das respostas.

6.6. REQUESTS

Biblioteca Python utilizada pelo cliente para realizar requisições HTTP à API desenvolvida com Flask.

6.7. GIT E GITHUB

Ferramentas utilizadas para versionamento do código, organização das atividades, colaboração entre integrantes e disponibilização pública do projeto.

7. ARQUITETURA DO SISTEMA

O TechGamer utiliza uma arquitetura cliente-servidor, organizada em três partes principais:

CLIENTE — QT (PYSIDE6)
Responsável pela interação com o usuário, apresentação das telas e envio de solicitações ao servidor.

SERVIDOR — FLASK
Responsável pelo processamento das solicitações, validação de dados, gerenciamento das operações e comunicação com o armazenamento.

Persistência — JSON
Responsável pela gravação e recuperação das informações em arquivos locais.

FLUXO DE FUNCIONAMENTO

1. O usuário realiza uma ação na interface gráfica.
2. O cliente Qt envia uma requisição HTTP ao servidor Flask.
3. O servidor recebe e valida os dados.
4. O servidor consulta ou atualiza o armazenamento JSON.
5. O servidor retorna uma resposta ao cliente.
6. A interface apresenta o resultado ao usuário.

EXEMPLO: CADASTRO DE JOGO

O usuário preenche o formulário de cadastro e clica no botão para salvar.

A interface envia os dados ao servidor Flask, que verifica as informações e registra o novo jogo por meio do módulo de persistência JSON.

Após a operação, o servidor retorna o resultado, e a interface apresenta a confirmação ao usuário.

8. REQUISITOS FUNCIONAIS

Os requisitos funcionais representam as operações e funcionalidades disponibilizadas pelo sistema TechGamer.

RF01 — Cadastrar jogos: permitir o registro de novos jogos.

RF02 — Listar jogos: apresentar os jogos cadastrados.

RF03 — Consultar jogo: permitir visualizar as informações de um jogo específico.

RF04 — Editar jogo: permitir modificar as informações de um jogo existente.

RF05 — Excluir jogo: permitir remover um jogo cadastrado e suas avaliações associadas.

RF06 — Cadastrar avaliação: permitir registrar uma nota e comentar um jogo.

RF07 — Listar avaliações: apresentar as avaliações associadas a um jogo.

RF08 — Editar avaliação: permitir alterar a nota e o comentário de uma avaliação.

RF09 — Excluir avaliação: permitir remover uma avaliação.

RF10 — Validar informações: verificar os campos enviados pelo usuário.

RF11 — Armazenar informações: salvar jogos e avaliações em arquivos JSON.

RF12 — Recuperar informações: carregar os dados armazenados ao reiniciar o sistema.

RF13 — Exibir mensagens: informar resultados de operações e erros encontrados.

RF14 — Pesquisar e filtrar jogos: permitir localizar jogos por meio de pesquisa e filtros.

RF15 — Ordenar e paginar jogos: permitir organizar e navegar pelos jogos cadastrados.

RF16 — Gerenciar perfis locais: permitir criar, selecionar e alternar perfis localmente.

RF17 — Consultar médias: calcular e apresentar a média das avaliações de cada jogo.

RF18 — Gerenciar minhas avaliações: permitir consultar, pesquisar, editar e excluir avaliações associadas ao perfil local, respeitando as permissões.

9. REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais estabelecem as características técnicas, as restrições e as condições necessárias para o funcionamento do TechGamer.

RNF01 — Linguagem: o sistema deve ser desenvolvido em Python.

RNF02 — Servidor: o sistema deve utilizar Flask para processamento das operações.

RNF03 — Interface: o cliente deve possuir interface gráfica desenvolvida com Qt.

RNF04 — Armazenamento: os dados devem ser armazenados localmente em arquivos JSON, sem utilização de banco de dados.

RNF05 — Comunicação: o cliente deve consumir os endpoints HTTP disponibilizados pelo servidor Flask.

RNF06 — Usabilidade: a interface deve apresentar navegação clara, formulários compreensíveis e mensagens informativas.

RNF07 — Persistência: os dados devem permanecer disponíveis após o encerramento e a reinicialização do aplicativo.

RNF08 — Tratamento de erros: o sistema deve informar operações inválidas, falhas de comunicação e erros de processamento.

RNF09 — Organização: o código deve estar estruturado em módulos com responsabilidades definidas.

RNF10 — Execução: o aplicativo deve funcionar no ambiente de entrega sem necessidade de instalação adicional de Python ou de suas dependências.

RNF11 — Restrições: o sistema não deve utilizar bancos de dados, serviços de nuvem ou APIs externas para o gerenciamento dos jogos e avaliações.

RNF12 — Funcionamento local: as operações principais devem ocorrer por meio da comunicação entre o cliente e o servidor local.

10. REGRAS DE NEGÓCIO

As regras de negócio determinam as condições que devem ser respeitadas durante as operações do sistema.

RN01: nome, gênero, plataforma e ano de lançamento são campos obrigatórios.

RN02: a descrição e a capa do jogo são opcionais.

RN03: o ano de lançamento deve ser um número inteiro entre 1950 e o ano atual.

RN04: cada jogo possui um identificador único gerado automaticamente pelo sistema.

RN05: uma avaliação deve estar associada a um jogo existente.

RN06: a nota da avaliação deve estar entre 0 e 10, incluindo valores decimais.

RN07: o comentário da avaliação é opcional.

RN08: cada avaliação possui um identificador único.

RN09: um jogo pode possuir diversas avaliações.

RN10: ao excluir um jogo, suas avaliações associadas também são removidas.

RN11: o sistema utiliza perfis locais identificados por números positivos, sem exigir autenticação por login e senha.

RN12: a edição e a exclusão individual de avaliações são permitidas ao autor identificado ou ao moderador configurado para demonstração.

RN13: a média de um jogo é calculada a partir das notas de suas avaliações.

RN14: os dados são armazenados localmente, sem sincronização automática pela internet.

RN15: as informações cadastradas devem permanecer disponíveis após a reinicialização do aplicativo.

11. COMUNICAÇÃO ENTRE CLIENTE E SERVIDOR

O TechGamer utiliza requisições HTTP para estabelecer a comunicação entre a interface Qt e o servidor Flask.

O servidor disponibiliza endpoints responsáveis pelo cadastro, consulta, atualização e exclusão de jogos e avaliações, além de consultas de médias e avaliações por autor.

Método	Endpoint	Finalidade
GET	`/health`	Verificar o servidor
GET	`/jogos`	Listar jogos
GET	`/jogos/<id>`	Consultar jogo
POST	`/jogos`	Cadastrar jogo
PUT	`/jogos/<id>`	Editar jogo
DELETE	`/jogos/<id>`	Excluir jogo
GET	`/jogos/<id>/avaliacoes`	Listar avaliações
POST	`/jogos/<id>/avaliacoes`	Cadastrar avaliação
PUT	`/avaliacoes/<id>`	Editar avaliação
DELETE	`/avaliacoes/<id>`	Excluir avaliação
GET	`/jogos/<id>/media`	Consultar média do jogo
GET	`/autores/<autor_id>/avaliacoes`	Consultar avaliações de um autor

A API também permite consultar o catálogo com médias por meio do parâmetro `com_medias=1` na rota `/jogos`.

As respostas são retornadas em formato JSON, permitindo que a interface apresente os resultados das operações ao usuário.

Os formatos dos dados enviados e recebidos estão detalhados no arquivo `documentacao/contrato-api.md`, disponível no repositório.

12. PADRÕES DE PROJETO (DESIGN PATTERNS)

O TechGamer utiliza três padrões de projeto: Builder, Proxy e Observer, aplicados ao gerenciamento das avaliações.

12.1. BUILDER

O padrão Builder é utilizado na construção das avaliações, organizando e validando informações como autor, nota, comentário e metadados.

Sua implementação está localizada no módulo `servidor/patterns/avaliacoes.py`, por meio da classe `AvaliacaoBuilder`.

O objetivo é centralizar a construção das avaliações e garantir que seus dados estejam adequados antes do armazenamento.

12.2. PROXY

O padrão Proxy é utilizado para controlar as permissões de edição e exclusão individual de avaliações.

A classe `AvaliacoesProxy` verifica se o identificador informado corresponde ao autor da avaliação ou a um moderador autorizado.

Caso a identificação não tenha permissão, a operação é bloqueada.

Esse mecanismo demonstra o controle de acesso por meio de identificadores locais, sem representar um sistema de autenticação real.

12.3. OBSERVER

O padrão Observer é utilizado para atualizar as informações relacionadas às médias das avaliações.

Após o cadastro, a edição ou a exclusão de uma avaliação, o sistema notifica os observadores responsáveis pela atualização dessas informações.

As classes “EventosAvaliacoes” e “NotaMediaObserver” participam desse processo.

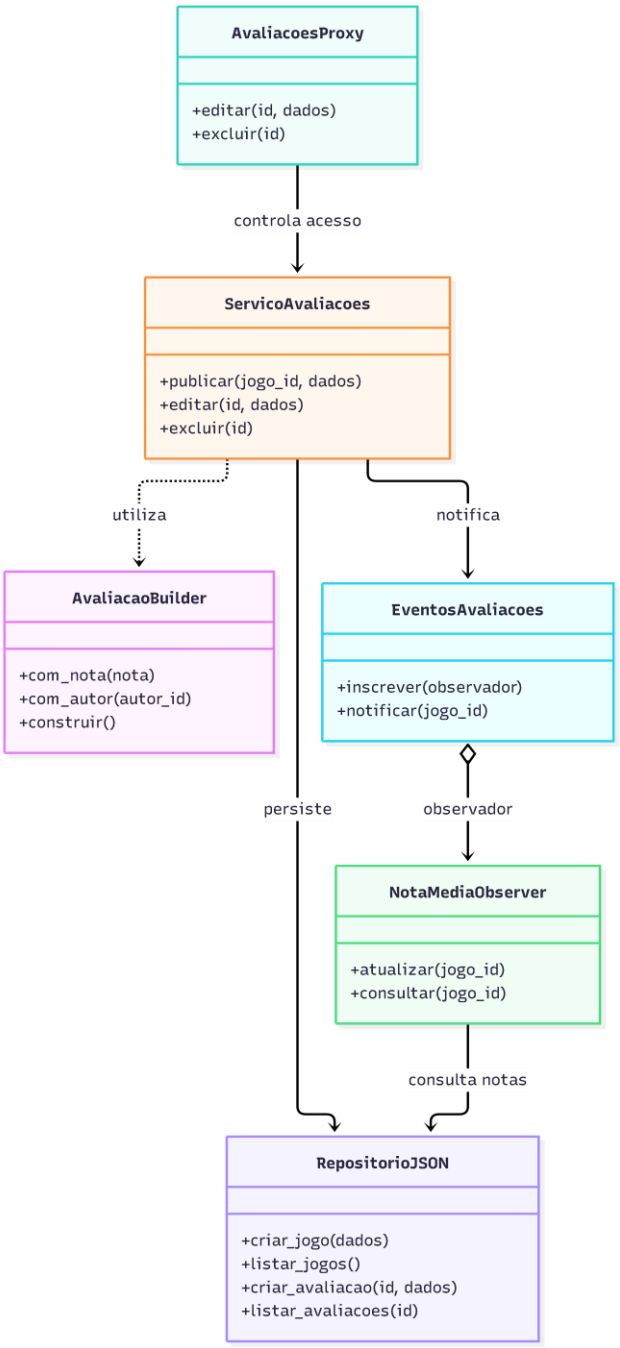
A aplicação desses padrões contribui para a organização do código, a separação de responsabilidades e a manutenção das funcionalidades.

13. DIAGRAMAS DE CLASSES

Os diagramas de classes representam a estrutura do TechGamer, destacando as principais classes, suas responsabilidades e seus relacionamentos.

13.1. DIAGRAMA DE CLASSES DO SERVIDOR

O diagrama mostra as principais classes responsáveis pelo processamento das avaliações, aplicação dos padrões de projeto e persistência das informações no servidor Flask.

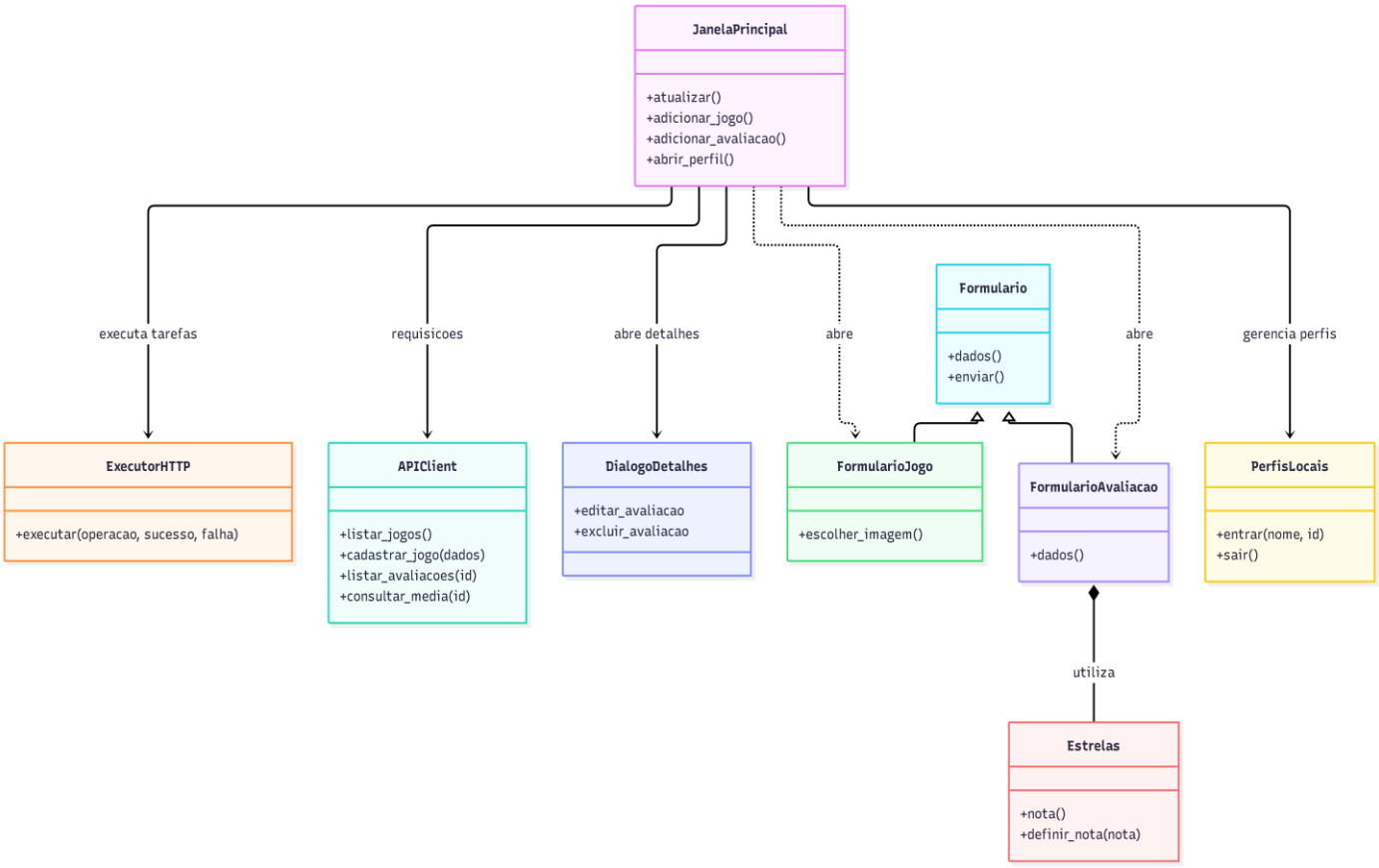

AvaliacoesProxy
+editar(id, dados)
+excluir(id)
controla acesso
ServicoAvaliacoes
+publicar(jogo_id, dados)
+editar(id, dados)
+excluir(id)
utiliza
notifica
AvaliacaoBuilder
+com_nota(nota)
+com_autor(autor_id)
+construir()
EventosAvaliacoes
+inscrever(observador)
+notificar(jogo_id)
persiste
observador
NotaMediaObserver
+atualizar(jogo_id)
+consultar(jogo_id)
consulta notas
RepositorioJSON
+criar_jogo(dados)
+listar_jogos()
+criar_avaliacao(id, dados)
+listar_avaliacoes(id)

13.2. DIAGRAMA DE CLASSES DO CLIENTE

O diagrama apresenta as principais classes responsáveis pela interface gráfica do TechGamer e suas funções:

- **JanelaPrincipal:** organiza as telas e funcionalidades do aplicativo.
- **Formulários:** permitem cadastrar e editar jogos e avaliações.
- **APIClient:** realiza a comunicação com o servidor Flask.
- **ExecutorHTTP:** executa operações sem travar a interface.
- **PerfisLocais:** gerencia os perfis utilizados nas avaliações.
- **Estrelas:** permite selecionar as notas dos jogos.
- **DialogoDetalhes:** apresenta as informações e avaliações dos jogos.

Essas classes trabalham em conjunto para garantir a interação do usuário com o sistema.



14. CONDIÇÕES DE ACEITAÇÃO

As condições de aceitação estabelecem os resultados esperados para verificar se as funcionalidades do TechGamer atendem aos requisitos definidos.

Funcionalidade	Condição de aceitação
Cadastro de jogo	Um jogo com dados válidos é registrado e aparece na biblioteca.
Consulta de jogo	As informações do jogo selecionado são apresentadas corretamente.
Edição de jogo	As alterações realizadas são salvas e exibidas.
Exclusão de jogo	O jogo é removido após confirmação, juntamente com suas avaliações.
Cadastro de avaliação	Uma nota válida é registrada para um jogo existente.
Consulta de avaliações	As avaliações cadastradas são apresentadas corretamente.
Edição de avaliação	As alterações da nota e do comentário são armazenadas quando autorizadas.
Exclusão de avaliação	A avaliação é removida quando a operação é autorizada.
Validação de nota	Notas fora do intervalo de 0 a 10 são rejeitadas.
Cálculo de média	A média apresentada corresponde às avaliações cadastradas.
Perfis locais	As avaliações podem ser consultadas conforme o identificador do perfil.
Controle de permissões	Operações de edição e exclusão individual sem autorização são bloqueadas.
Pesquisa e filtros	Os resultados apresentados correspondem aos critérios selecionados.
Persistência JSON	Os dados permanecem disponíveis após reiniciar o aplicativo.
Integração	As operações realizadas na interface são processadas pelo servidor Flask.
Inicialização	O aplicativo pode ser executado no Windows sem instalação adicional das dependências.

Essas condições são verificadas por meio dos testes automatizados e dos testes manuais realizados pela equipe.

15. PLANO E RESULTADOS DOS TESTES

Os testes foram realizados para verificar o funcionamento do TechGamer, a comunicação entre a interface e o servidor e o armazenamento das informações.

A execução automatizada registrada apresentou **107 testes aprovados, sem falhas ou erros**. As imagens a seguir demonstram o funcionamento das principais funcionalidades do aplicativo.

15.1. TESTES DO SERVIDOR

Foram verificados o cadastro, a consulta, a edição e a exclusão de jogos e avaliações, além da validação das informações.

Os testes foram executados pelo terminal do VS Code, utilizando os testes automatizados do servidor Flask.

Resultado: 28 testes executados e aprovados, sem falhas ou erros

.

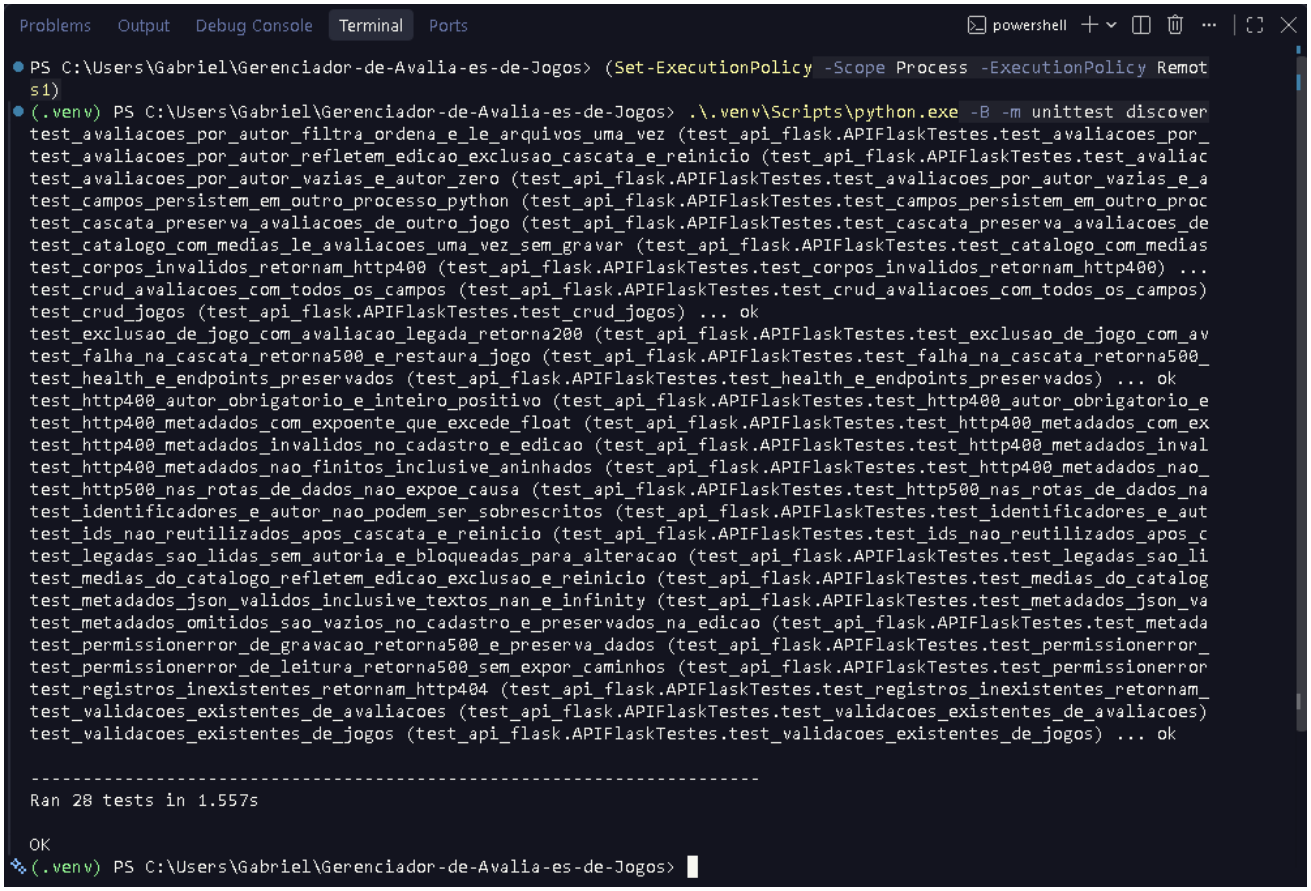


Figura 1 — Resultado dos testes automatizados do servidor Flask.

15.2. TESTES DE PERSISTÊNCIA

O teste foi realizado para verificar se as informações permanecem armazenadas após o encerramento e a reinicialização do aplicativo.

Informações verificadas:

- **Jogo:** Super Mario Bros. 3
- **Nota:** 9/10
- **Avaliações:** 1
- **Perfil:** Gabriel
- **Capa e descrição:** presentes



Figura 2 — Informações do jogo antes da reinicialização do TechGamer.

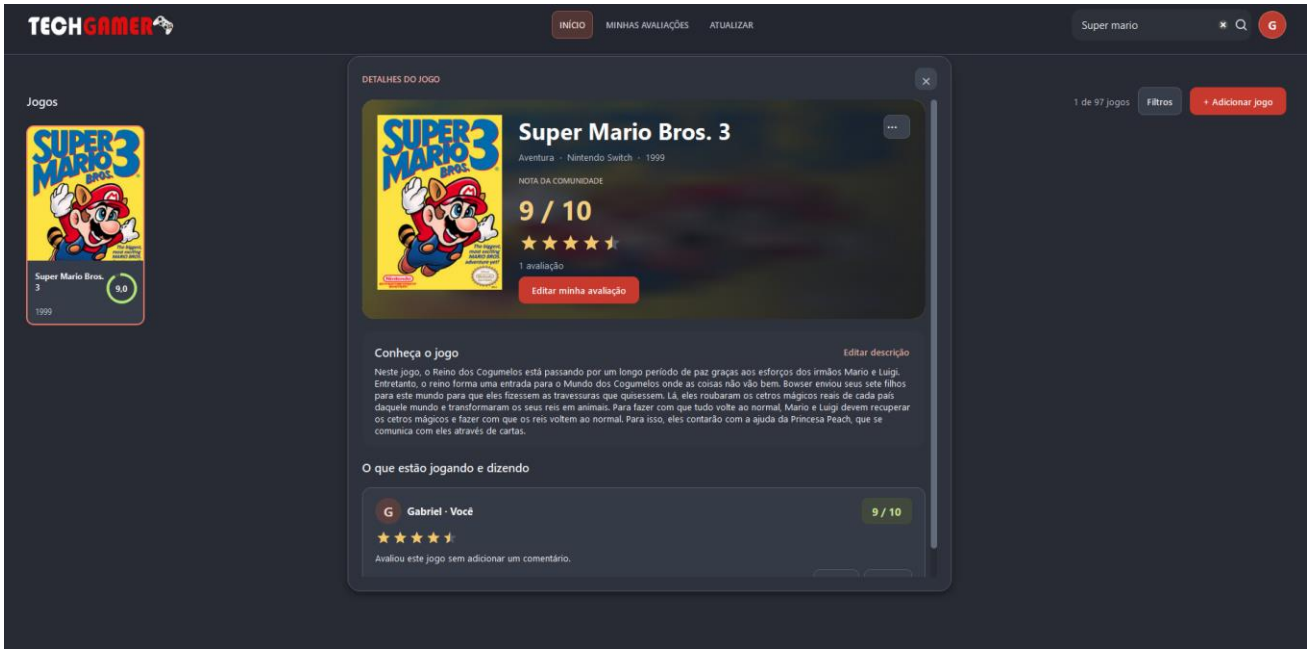


Figura 3 — Informações do jogo preservadas após a reinicialização do TechGamer.

Resultado: APROVADO. Após fechar e abrir novamente o TechGamer, o jogo e sua avaliação permaneceram disponíveis, comprovando a persistência dos dados armazenados em JSON.

15.3. TESTES DA INTERFACE

Os testes da interface têm como objetivo verificar o funcionamento das telas, dos formulários e das principais funcionalidades disponibilizadas ao usuário.

TESTE 01 — PESQUISA DE JOGOS

Foi realizada uma pesquisa pelo nome “Call of Duty”, para verificar se o sistema apresenta os jogos correspondentes.

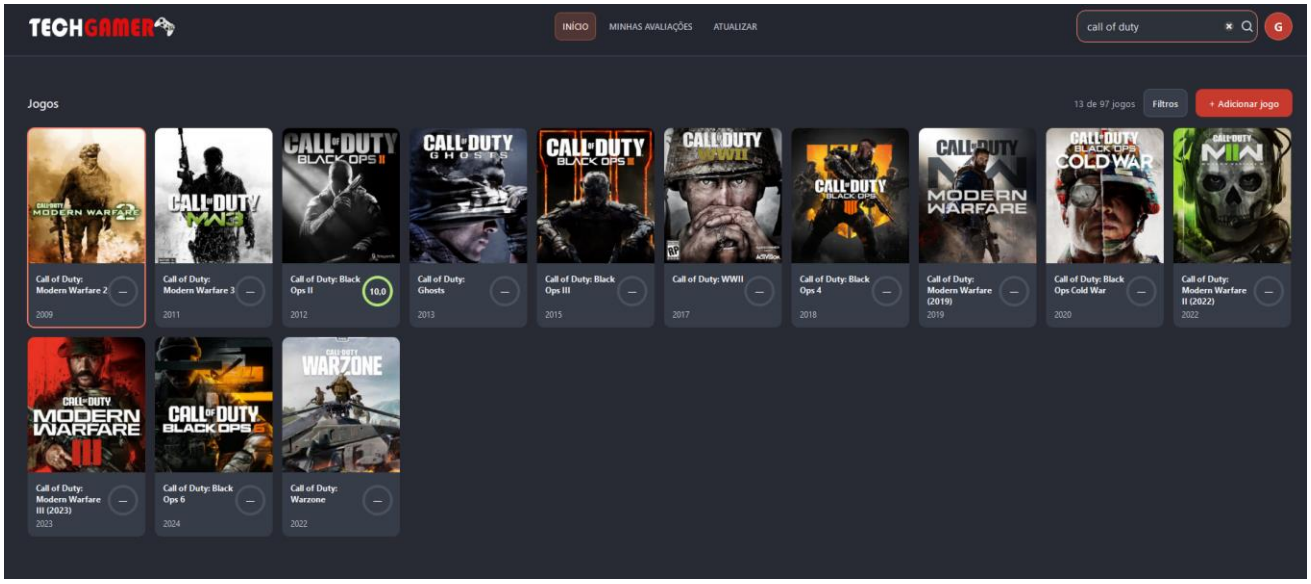


Figura 4 — Pesquisa de jogos na biblioteca do TechGamer.

TESTE 02 — FILTROS DE JOGOS

Foram aplicados filtros de gênero e plataforma, para verificar se os resultados apresentados correspondem aos critérios selecionados.

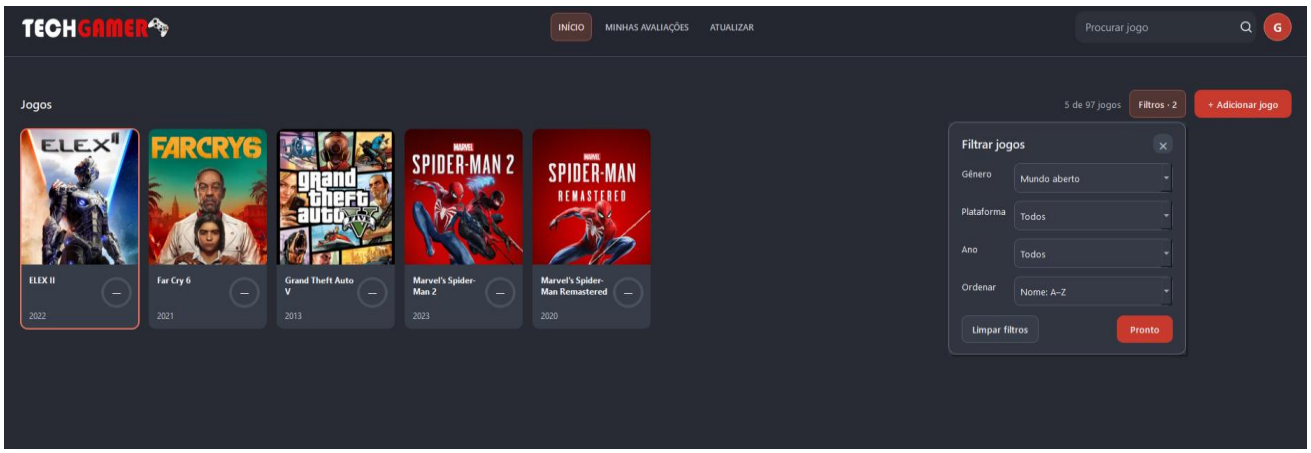


Figura 5 — Aplicação de filtros na biblioteca do TechGamer.

TESTE 03 — CADASTRO DE JOGO (DUAS FOTOS)

Foi realizado o cadastro de um novo jogo, preenchendo os campos obrigatórios e verificando sua apresentação na biblioteca.



Figura 6 — Formulário de cadastro de jogo preenchido no TechGamer.

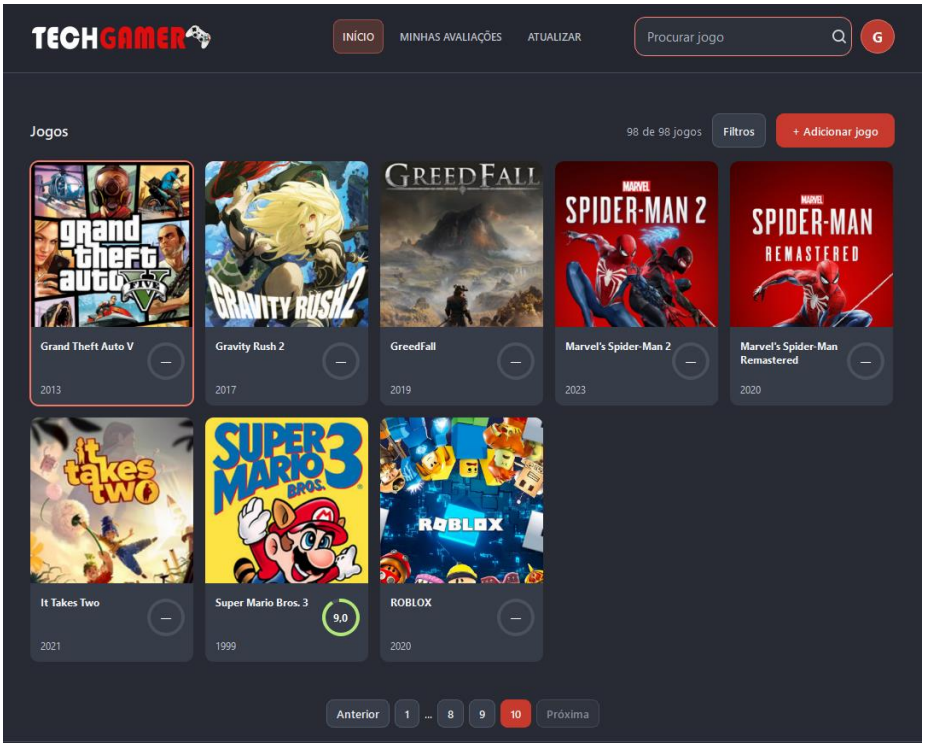


Figura 7 — Jogo cadastrado e apresentado na biblioteca do TechGamer.

TESTE 04 — VALIDAÇÃO DOS CAMPOS

Foi realizada uma tentativa de cadastro com informações incompletas, para verificar se o sistema impede o registro e apresenta uma mensagem informativa.

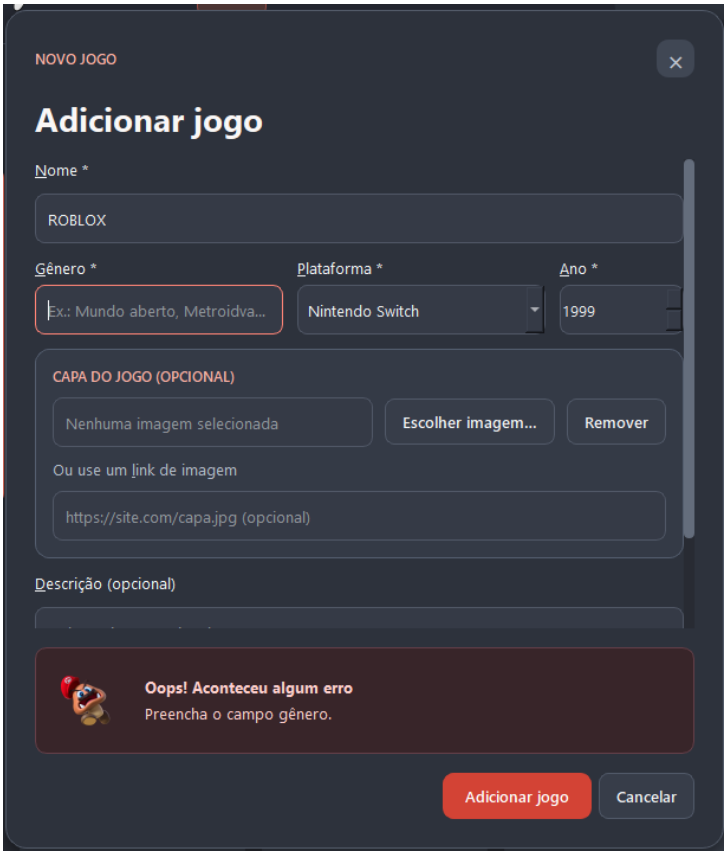


Figura 8 — Mensagem de validação apresentada ao tentar cadastrar um jogo com informações incompletas.

TESTE 05 — EDIÇÃO DE JOGO

Foi realizada uma alteração nas informações de um jogo cadastrado, para verificar se os novos dados são apresentados corretamente.

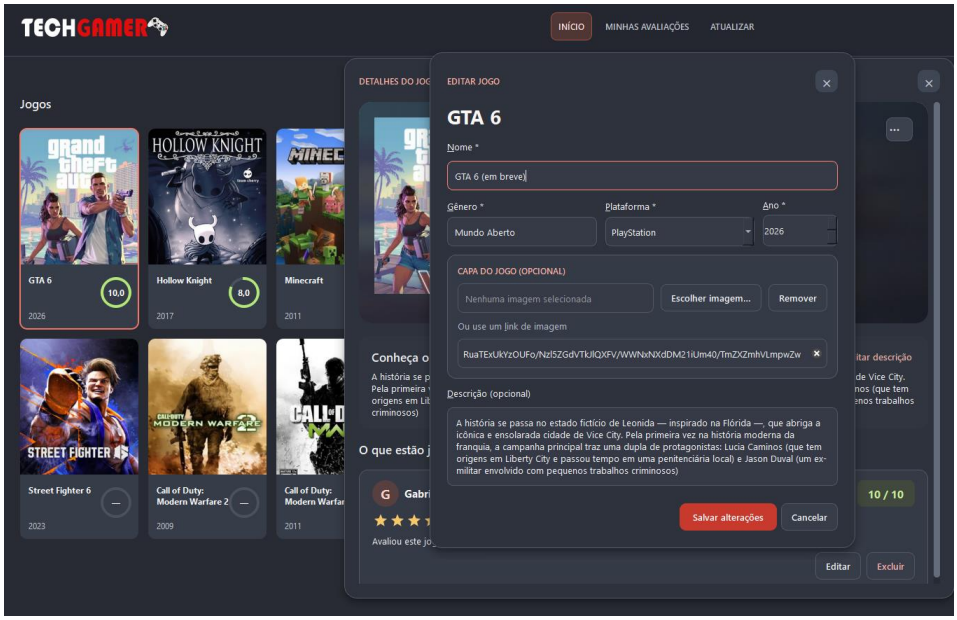


Figura 9 — Formulário de edição das informações de um jogo.

TESTE 06 — CADASTRO DE AVALIAÇÃO (DUAS FOTOS)

Foi realizada uma avaliação de um jogo cadastrado, atribuindo uma nota e adicionando um comentário, para verificar se as informações são registradas corretamente e se a média do jogo é atualizada.

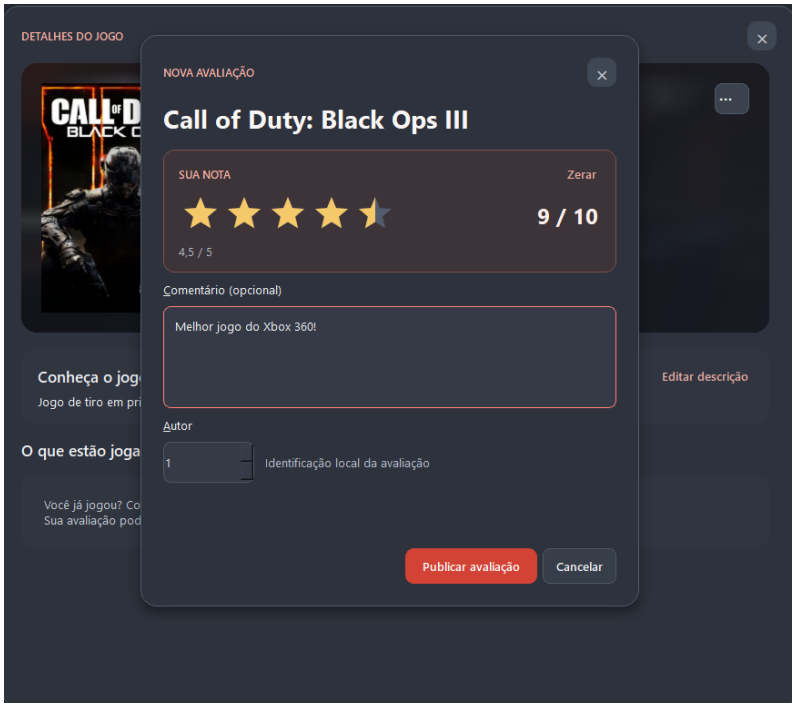


Figura 10 — Formulário de cadastro de avaliação com nota e comentário.

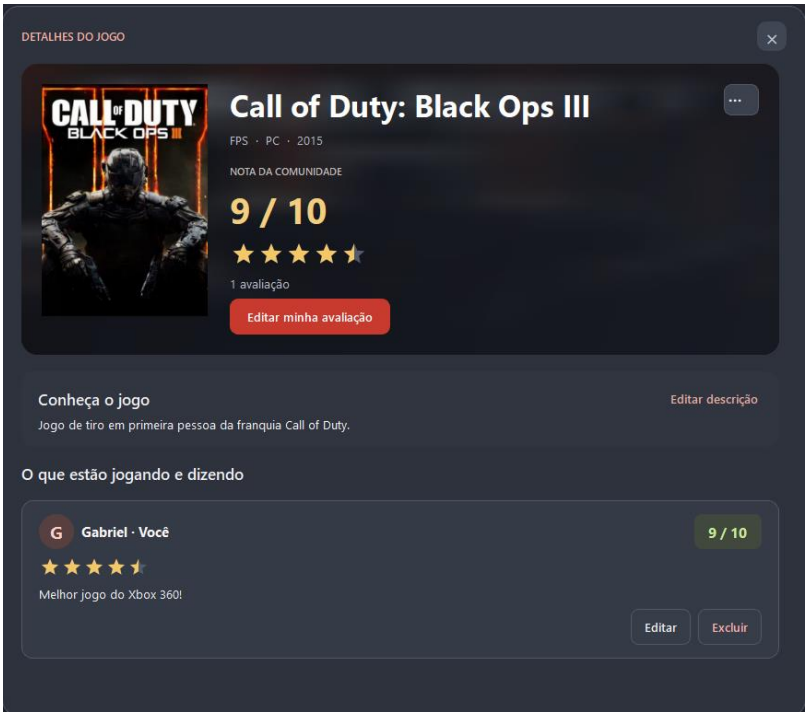


Figura 11 — Avaliação publicada e apresentada nos detalhes do jogo.

TESTE 07 — EDIÇÃO DE AVALIAÇÃO (DUAS FOTOS)

Foi realizada uma alteração na nota e no comentário de uma avaliação existente, para verificar se as novas informações são atualizadas e apresentadas corretamente.

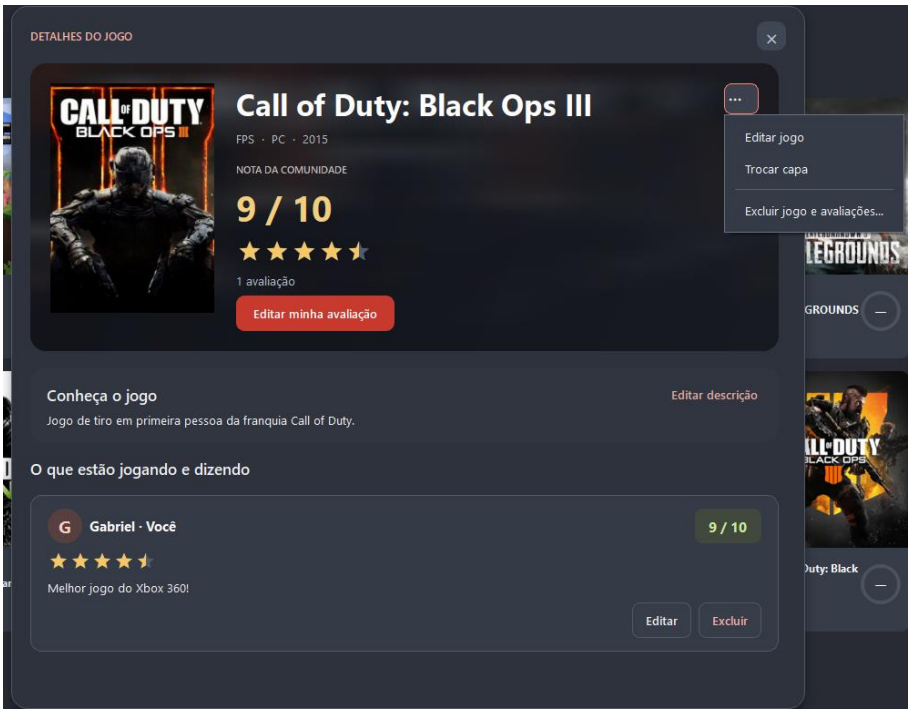


Figura 12 — Avaliação existente com opções de gerenciamento.

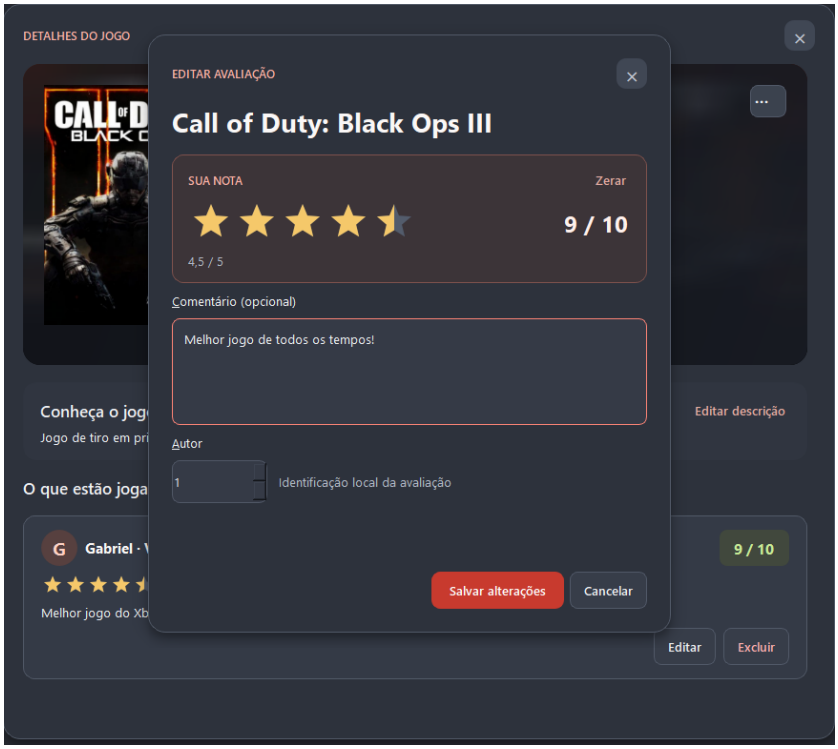


Figura 13 — Formulário de edição da nota e do comentário de uma avaliação.

TESTE 08 — MINHAS AVALIAÇÕES

Foi acessada a seção “Minhas avaliações”, para verificar se o aplicativo apresenta as avaliações vinculadas ao perfil selecionado.

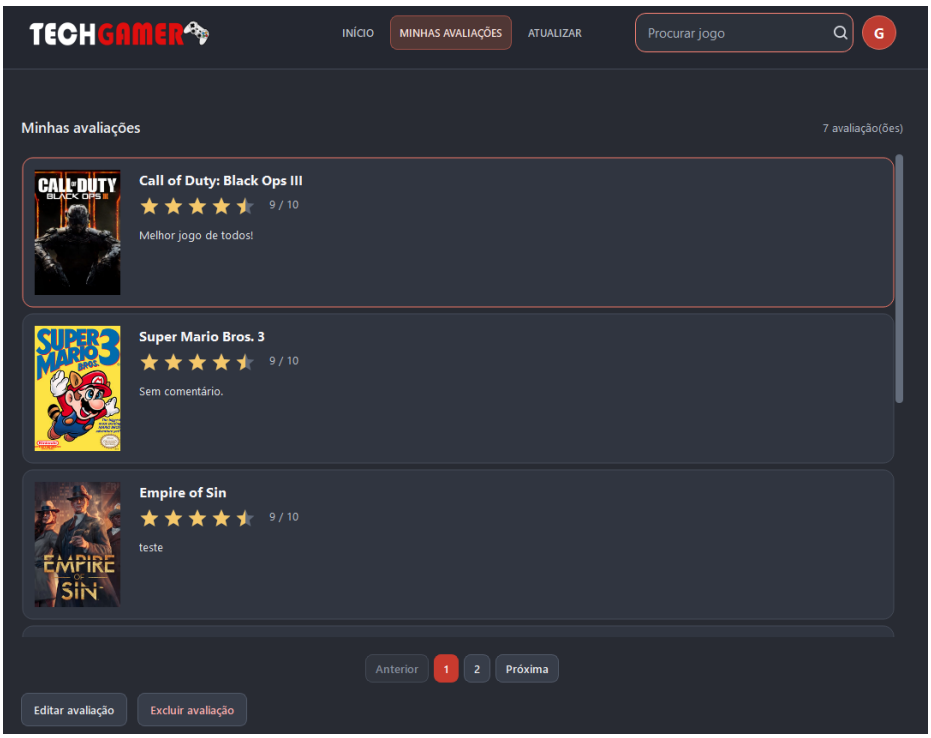


Figura 14 — Consulta das avaliações vinculadas ao perfil no TechGamer.

TESTE 09 — EXCLUSÃO DE REGISTROS (DUAS FOTOS)

Foi realizada a exclusão de um registro de teste, para verificar se o sistema solicita confirmação e remove as informações corretamente.

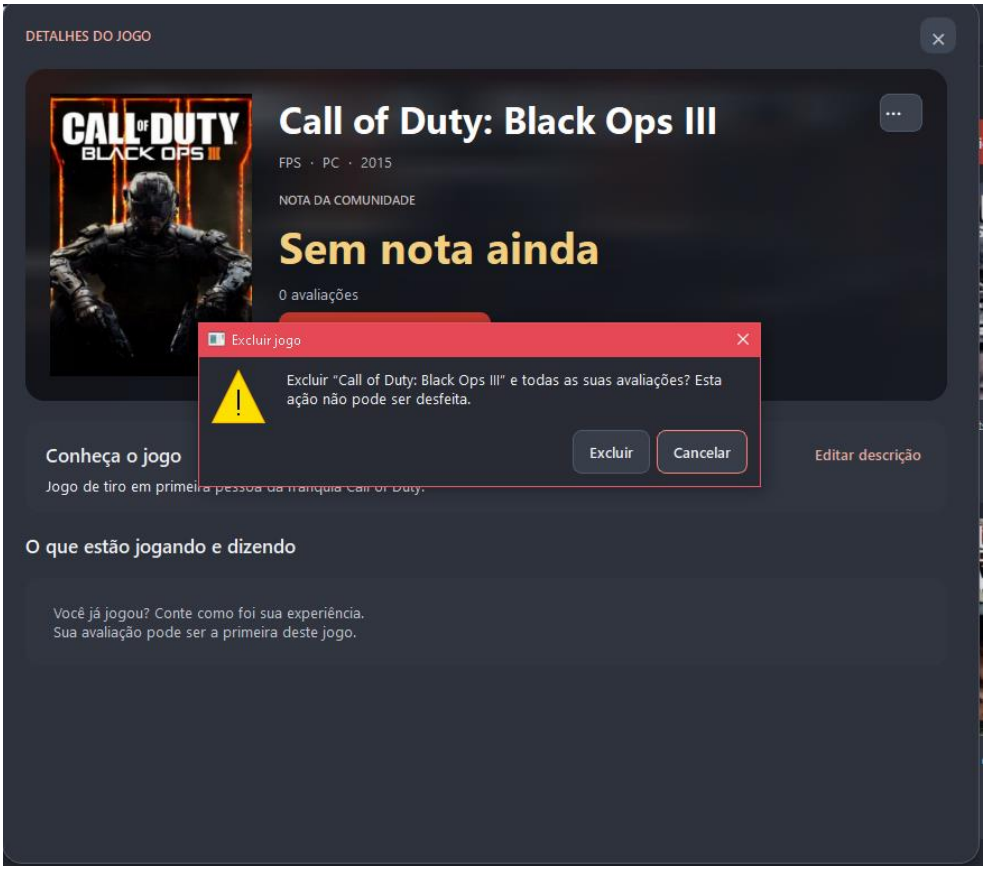


Figura 15 — Mensagem de confirmação para exclusão de um jogo.

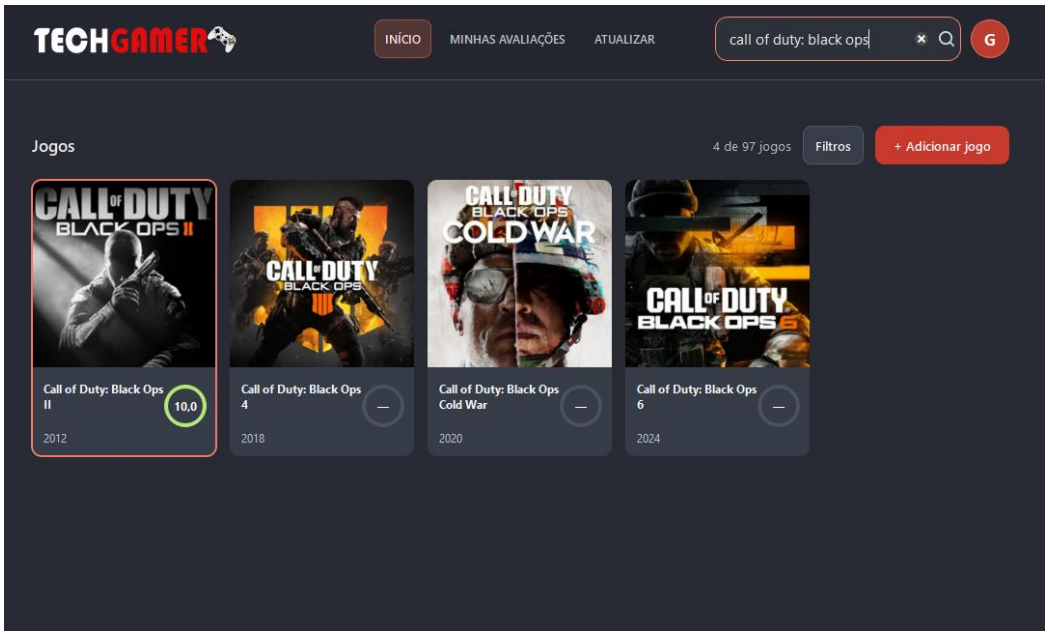


Figura 16 — Biblioteca após a exclusão do jogo selecionado.

TESTE 10 — NAVEGAÇÃO E MENSAGENS

Foi verificada a navegação entre as telas do aplicativo e a apresentação de mensagens de confirmação e erro durante as operações realizadas.

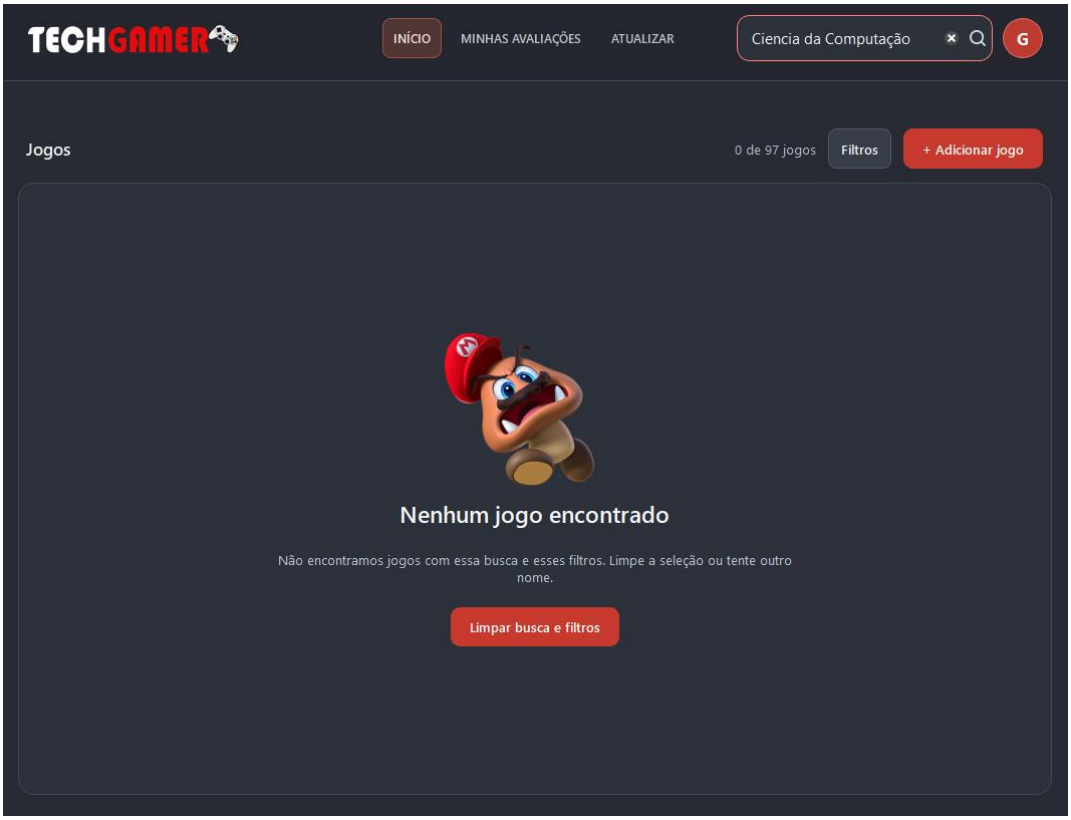


Figura 17 — Mensagem informativa apresentada quando a pesquisa não encontra jogos.

RESULTADO GERAL

Resultado: APROVADO. Os testes realizados demonstraram o funcionamento das principais funcionalidades da interface do TechGamer, incluindo pesquisa, filtros, cadastro, edição e exclusão de jogos e avaliações, consulta das avaliações do perfil e apresentação de mensagens informativas.

As operações verificadas apresentaram os resultados esperados, sem falhas identificadas durante os testes manuais.

16. INSTRUÇÕES DE EXECUÇÃO

O TechGamer foi disponibilizado em uma versão portátil para computadores com Windows de 64 bits.

O pacote contém o executável e as dependências necessárias para iniciar o aplicativo, sem exigir a instalação separada de Python, Flask ou Qt.

16.1. ARQUIVOS NECESSÁRIOS

A versão portátil contém:

- `TechGamer.exe`: executável principal.
- `_internal`: arquivos e bibliotecas necessários ao funcionamento.
- `dados`: arquivos utilizados para armazenar jogos e avaliações.
- `LEIA-ME.txt`: orientações básicas de utilização.

16.2. COMO INICIAR O APLICATIVO

1. Baixar o pacote portátil disponibilizado no GitHub.
2. Extrair o conteúdo do arquivo ZIP.
3. Abrir a pasta `TechGamer`.
4. Executar o arquivo `TechGamer.exe`.
5. Aguardar a abertura da interface gráfica.

O servidor Flask é iniciado automaticamente pelo aplicativo, estabelecendo a comunicação local com a interface Qt.

16.3. ARMAZENAMENTO DAS INFORMAÇÕES

Os jogos e as avaliações são armazenados em arquivos JSON dentro da pasta `dados`.

As informações permanecem disponíveis após o encerramento do programa, permitindo que o usuário continue utilizando sua biblioteca ao abrir novamente o aplicativo.

Para manter os dados ao transferir a aplicação para outro computador, é necessário preservar a pasta do programa, incluindo os arquivos de armazenamento.

16.4. REQUISITOS PARA EXECUÇÃO

- Computador com Windows de 64 bits.
- Permissão para executar o aplicativo.
- Pasta extraída em um local que permita a gravação dos dados.

As principais funcionalidades são executadas localmente, sem necessidade de conexão com a internet. Entretanto, capas de jogos cadastradas por links externos podem depender da disponibilidade desses endereços.

O código-fonte, a documentação e o pacote de entrega estão disponibilizados no repositório do projeto no GitHub.

17. CONCLUSÃO

O desenvolvimento do TechGamer permitiu criar um aplicativo desktop para organizar jogos eletrônicos e registrar avaliações pessoais de maneira prática.

O sistema reúne funcionalidades de cadastro, consulta, edição e exclusão de jogos e avaliações, além de pesquisa, filtros, perfis locais e cálculo das médias.

Durante o desenvolvimento, foram aplicados conceitos de programação em Python, arquitetura cliente-servidor, comunicação HTTP, persistência em JSON e os padrões de projeto Builder, Proxy e Observer.

A integração entre Flask e Qt (PySide6) possibilitou a construção de uma aplicação com interface gráfica e armazenamento local das informações.

Os testes automatizados registraram 107 aprovações, sem falhas ou erros. Além disso, a execução do aplicativo em outro computador demonstrou o funcionamento da versão portátil, sem necessidade de instalação adicional de Python.

Dessa forma, o TechGamer atingiu os objetivos propostos, reunindo os conhecimentos adquiridos durante a disciplina em uma aplicação funcional e organizada.